1 Overview

This release contains a co-registered in-situ and ex-situ *Peregrine* dataset from a single Arcam Q10 Electron Beam Powder Bed Fusion (EB-PBF) Inconel 738 build. These data were collected at the Manufacturing Demonstration Facility (MDF) located at Oak Ridge National Laboratory (ORNL). The dataset includes layer-wise Near Infrared (NIR) in-situ imaging data, in-situ temporal sensor data, ex-situ X-Ray Computed Tomography (X-CT) scans, and the target part geometries. Additionally, anomaly detections produced by a trained Dynamic Segmentation Convolutional Neural Network (DSCNN) [1] are provided. Figure 1 shows example slices of each of these data modalities for a single print layer. Download instructions and a full description of the dataset structure are provided in the following sections.

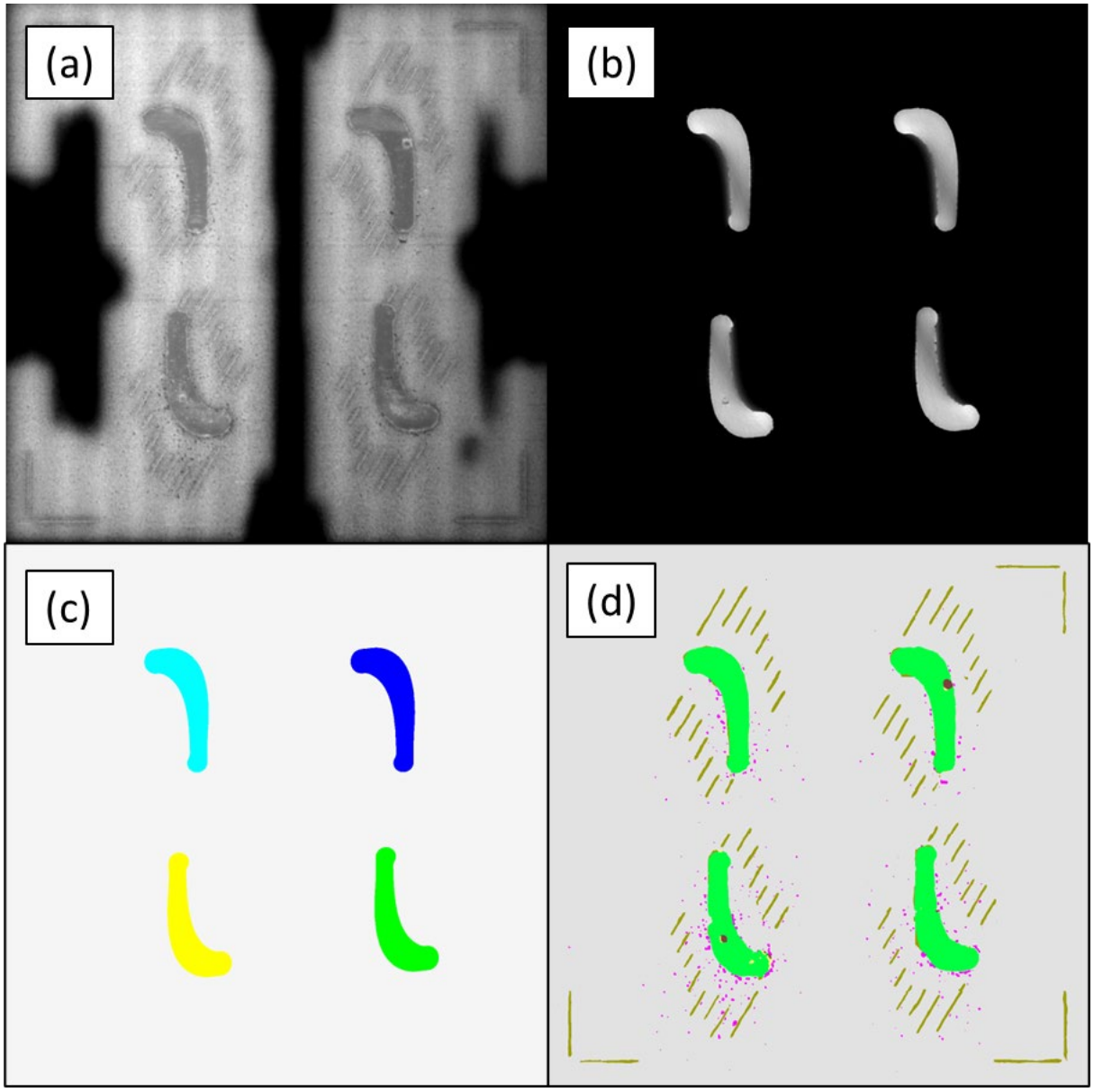

Figure 1. Co-registered data slices from layer 1408, approximately 70.40 mm into the build height. (a) An in-situ NIR image captured after the powder has been melted. (b) Reconstructed ex-situ X-CT scans for all four printed parts. (c) The intended STL geometries for each turbine blade, colored by their part ID. (d) Anomaly segmentations produced by a trained DSCNN with each color representing a different anomaly class.

2 Dataset Download Instructions

1. Create a Globus account.
2. Create a Globus Endpoint on your computer. You may need to create an exception for Globus in your antivirus software so that it can create an Endpoint.
3. Transfer the dataset from the OLCF DOI-DOWNLOADS Collection to your Collection. Be sure to confirm that the transfer is going from OLCF DOI-DOWNLOADS to your Collection.
4. Sometimes users will need to manually create a Globus access directory (where the data will be downloaded) by going to the **Preferences > Access** tab before the download will begin.
5. Please direct any questions about accessing the dataset to Dr. Luke Scime scimelr@ornl.gov.

3 Dataset Organization

Unlike previous *Peregrine* dataset releases, this dataset is stored as a single HDF5 file. HDF5 is a common data format [2] optimized for working with large volumes of data and will be used by the *Peregrine* team for all planned future dataset releases. HDF5 files are hierarchical and self-describing, i.e., the data can be navigated similarly to a folder directory and metadata are attached to each datum. The released *Peregrine* data can be grouped into four categories based on how they are intended to be retrieved from the dataset. Users can explore the entire dataset using the *ExampleHDF5Open.py* Python 3.9 script provided alongside this dataset. Code snippets are also provided throughout this section for reference.

3.1 Build Metadata

Metadata applying to the entire build are stored as their native data type and can be accessed using their attribute key. The code snippet below accesses and prints out the **core/build_name** value, while the following list describes the contents of each high-level grouping of metadata. If available, units for a given piece of metadata can be accessed by appending **/units** to the attribute key.

```
import h5py
with h5py.File(path_to_hdf5_file,'r') as build:
    print(build.attrs['core/build_name'])
```

- **core/:** Contains information uniquely identifying this build including the name and any notes.
- **log_file/:** This blob of text is extracted directly from the log file produced by Arcam Q10 printers.
- **material/:** Contains information about the material composition and layer thickness.
- **people/:** Contains information about the people who requested and manufactured the parts.
- **printer/:** Describes the printer used to manufacture the build including its physical dimensions.
- **specimens/:** Contains information about the printed parts and any excised samples.
- **user_defined/:** Contains user-defined metadata fields that are typically printer-specific.

3.2 Reference Images

Reference images captured by the printer operator before, during, and after the build are stored as two-dimensional arrays and can be accessed by name and displayed using Matplotlib. The code snippet below displays the automatically generated **reference_images/thumbnail** which shows a two-dimensional projection of the three-dimensional part layout for this build.

```
import h5py
import matplotlib.pyplot as plt
with h5py.File(path_to_hdf5_file,'r') as build:
    plt.imshow(build['reference_images/thumbnail'][...],interpolation='none')
```

3.3 Slice Data

Slice data include the in-situ NIR images, the ex-situ X-CT reconstructions, the connectivity maps for the intended part geometries, and the DSCNN anomaly predictions. Each sliced data modality is stored as a two-dimensional array and indexed by the layer number. All of the sliced data are in the same global coordinate system and have been resized to the pixel resolution of the NIR data. The exception is the X-CT data which are provided at their native, higher pixel resolution. Information about the coordinate system origin and axes are saved as metadata in the format **slices/origin**, **slices/x-axis**, etc. The code snippet below displays an in-situ NIR image **slices/camera_data/NIR/0** captured immediately after the layer has been melted. The remaining slice-type data are described in the following list.

```
import h5py
import matplotlib.pyplot as plt
with h5py.File(path_to_hdf5_file,'r') as build:
    plt.imshow(build['slices/camera_data/NIR/0'][layer_number,...],
               interpolation='none')
```

- **camera_data/NIR/0:** Thes are in-situ images captured using a 5 MP NIR camera immediately after each layer is melted by the electron-beam. If the image is missing for a particular layer, the array will contain all zero values.
- **camera_data/NIR/1:** These are in-situ images captured using a 5 MP NIR camera immediately after a new layer of powder is spread across the powder bed. If the image is missing for a particular layer, the array will contain all zero values.
- **slices/part_ids:** Each printed part is automatically assigned a unique ID based on its location in the print volume. The corresponding part ID is stored for each pixel within the build volume, with an ID of zero representing regions outside of the intended print geometries.
- **slices/registered_data/x-ray_ct:** These are composite images of the ex-situ X-Ray CT reconstructions for each of the four printed parts. Note that while these scans have been registered to the global coordinate system, they are stored at a higher pixel resolution than the in-situ imaging data or the part IDs.
- **slices/segmentation_results:** The pixel-wise anomaly predictions produced by the trained DSCNN model are stored as an array of enumerated values. That is, an integer at each pixel indicates which anomaly type or class was predicted for that pixel. The corresponding enumerated list of classes can be accessed with using the **slices/segmentation_results/class_names** key. Brief descriptions of each class are provided in the following list. Segmentation results for layers with missing camera have data have been assigned values of 254.
  - **[0] Powder:** Areas of the powder bed which contain no detected anomalies or printed parts.
  - **[1] Printed:** Printed regions with no detected anomalies.
  - **[2] Porosity:** Surface-visible lack-of-fusion porosities.
  - **[3] Spatter:** Ejecta from the weld pool which has landed back on the powder bed.
  - **[4] Crack:** Macroscopic cracking, typically caused by thermal stresses.
  - **[5] Recoater Streaking:** Occurs if the powder rake is damaged or dragging a large particle.
  - **[6] Balling:** Overheating of the printed material that results in a spherical slag particle.
  - **[7] Swelling:** A distortion or warping of the printed material.
  - **[8] Super-Elevation:** A lack of powder coverage over top of a printed region.
  - **[9] Dimpling:** Local pitting often caused by poor powder spreading or keyholing porosity.
  - **[10] Misprint:** Printed material detected outside of the intended part geometries.

3.4 Temporal Data

Temporal data are sourced from the log file produced by Arcam Q10 printers and generally contain information regarding the health of the printer itself. Each temporal data modality is stored as a one-dimensional array and indexed by the layer number. If multiple measurements for the same sensor were captured during a single layer, they have been averaged together. The code snippet below displays a scatter plot of the **temporal/bottom_temperature_1** sensor measurement. Units for each temporal data modality can be retrieved by appending **/units** to the corresponding attribute key. The following list briefly describes each temporal data modality.

```
import h5py
import matplotlib.pyplot as plt
with h5py.File(path_to_hdf5_file,'r') as build:
    plt.scatter(np.arange(build['temporal/bottom_temperature_1'].shape[0]),
                build['temporal/bottom_temperature_1'])
```

- **temporal/backing_vacuum_gauge_fb:** Vacuum pressure in the backing pump.
- **temporal/beam_current:** The average electron-beam current.
- **temporal/bottom_temperature_1:** Thermocouple temperature recorded below the start plate.
- **temporal/build_time:** Cumulative build time.
- **temporal/chamber_vacuum_gauge_fb:** Vacuum pressure in the build chamber.
- **temporal/column_vacuum_gauge_fb:** Vacuum pressure in the electron-beam column.
- **temporal/differential_pressure_over_filter:** Differential pressure across the filter.
- **temporal/extra_temperature_1:** Thermocouple temperature recorded at an arbitrary location.
- **temporal/layer_times:** The duration of each print layer.
- **temporal/rake_left_sensor_pulse:** Measures the powder dosing.
- **temporal/rake_left_sensor_pulse_2:** Measures the powder dosing.
- **temporal/rake_right_sensor_pulse:** Measures the powder dosing.
- **temporal/rake_right_sensor_pulse_2:** Measures the powder dosing.
- **temporal/shunt_valve_water_mix:** Relates to the water flow through the chiller.
- **temporal/smoke_detector_counts:** Detections of powder smoking events.
- **temporal/table_current:** Measures the current at the start plate.
- **temporal/turbopump_[0]_current:** Current drawn by the vacuum turbopump.
- **temporal/turbopump_[1]_current:** Current drawn by the vacuum turbopump.

4 References

[1] L. Scime, D. Siddel, S. Baird, V. Paquit, Layer-wise anomaly detection and classification for powder bed additive manufacturing processes: A machine-agnostic algorithm for real-time pixel-wise semantic segmentation, Addit. Manuf. 36 (2020) 101453. https://doi.org/10.1016/j.addma.2020.101453.

[2] A. Collette, HDF5 for Python, (2014). https://docs.h5py.org/en/stable/ (accessed August 25, 2023).